



**Jaya Surya S**

Platform Engineer · Ford Motor Company

Containers



GPEN

KUBERNETES · GLOBAL PLATFORM ENGINEERS NETWORK · Q2 2026



# Gateway API

## The Future of Kubernetes Routing

Why Ingress is holding you back — and how Gateway API fixes it, scales it, and makes it production-grade.

Docker

Kuber



EXPLORE



01 / 07

DEPRECATED



## spec.externalIPs

`spec.externalIPs` on Services is deprecated. Future versions will require explicit opt-in via feature gate. Audit your manifests now to avoid broken routing after upgrades.

→ Audit Service manifests

02 / 07

REMOVED



## gitRepo Volume

`gitRepo` volume type is **fully removed** in v1.36. Deprecated since v1.11 (2018). Any pods still using `gitRepo` will fail to schedule on v1.36+ clusters. Migrate to `initContainers` with `git clone` as the replacement pattern.

→ Use `initContainers` + `git clone`

03 / 07

GA



## MutatingAdmissionPolicy

`MutatingAdmissionPolicy` reaches **GA** in v1.36. Declarative mutation of admission requests via CEL expressions — no webhook server needed. Replace custom mutating webhooks with in-process policies.

→ Replace mutating webhooks with CEL

04 / 07

GA



## SELinux Volume Relabeling

Recursive SELinux relabeling is now **stable**. Massive speedup for security-constrained pods at startup by eliminating the slow per-file relabel loop that blocked readiness.

→ Enable for security-constrained pods

05 / 07

GA



## User Namespaces

Linux user namespaces for pods is **stable** in v1.36. Containers run as UID 0 inside but map to unprivileged UIDs on the host — container breakout is no longer a root privilege escalation. Critical hardening for multi-tenant clusters.

→ Enable for multi-tenant security

06 / 07

BETA



## In-Place Pod Resize

Resize pod CPU and memory **without a restart** — now in Beta at pod level. Set `spec.resources` on a running pod and the kubelet applies it live. Eliminates noisy restarts for scaling ML inference or stateful workloads.

→ Live resize for stateful workloads

07 / 07

RETIRED



## ingress-nginx EOL

Kubernetes officially retired `ingress-nginx` on **March 24, 2026**. No more patches or security fixes. Clusters still running it are on unpatched, unsupported software. All teams must migrate to Gateway API — no extensions granted.

→ Migrate to Gateway API now

# Five Acts. One Revolution.



01

## The Annotation Graveyard Hiding in Your Cluster

Vendor lock-in, HTTP-only, no role separation — why Ingress doesn't fit modern clusters.

02

## What If No One Had to Own Everything?

Clean separation: infra, ops, and dev each own their layer — by design.

03

## One API to Route Them All

HTTPRoute · GRPCRoute · TLSRoute · Traffic Splitting · GAMMA — one spec, any controller.

04

## The Migration Nobody Should Delay

4-step cutover. Run old and new in parallel. One tool converts your Ingress YAML.

05

## Gateway API v1.5 Dropped — What's Inside

CORS, TLSRoute, ListenerSet GA. Six years of releases, one release timeline — what landed and why it matters.

# Ingress — Built for a Simpler World



## Poor Status & Observability

Status only surfaces one IP. No per-rule conditions, no attachment status, no lifecycle.



## Vendor Lock-in via Annotations

Custom annotations tie you to one controller. Your YAML breaks on any other implementation.



## HTTP/S Only by Design

TCP, UDP, and gRPC are not in the Ingress spec — non-standard extensions required.



## No Cross-Namespace Routing

Cross-namespace routing is unsupported by spec. Every workaround is controller-specific.



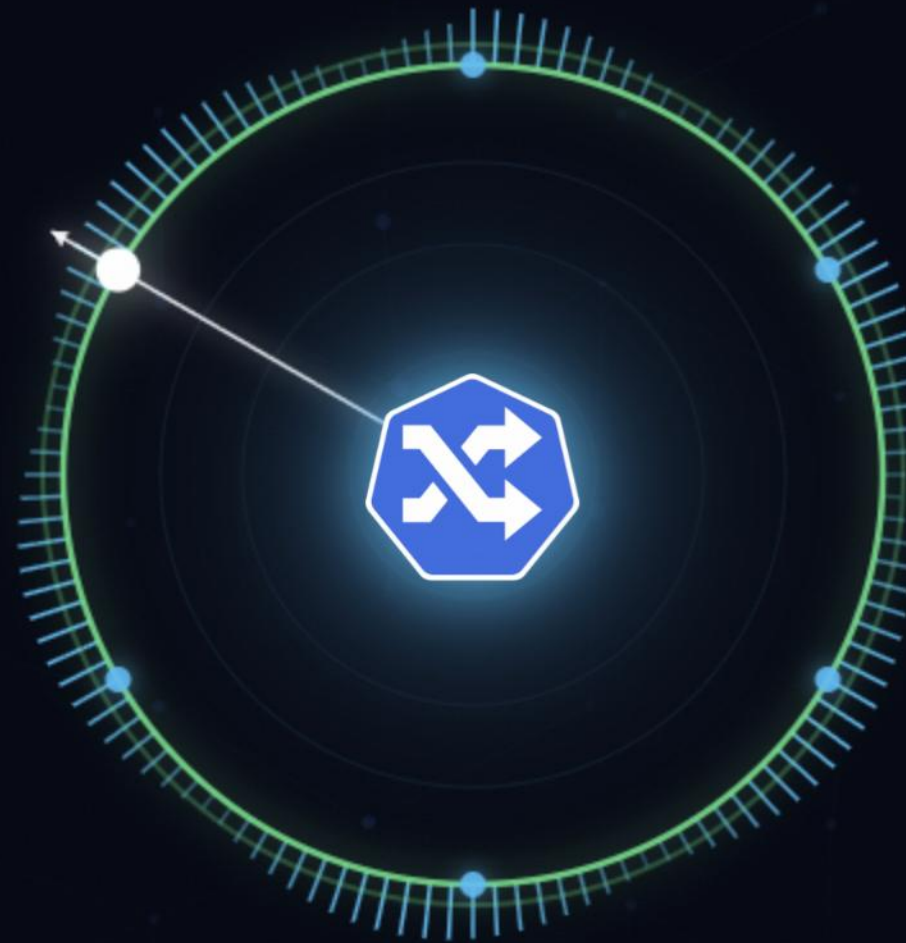
## No Role Separation

Ops and devs share the same resource. Safely delegating routing control is impossible.



## No Native Traffic Splitting

Canary and weighted routing are absent from spec. Every controller invents its own.



# Gateway API — Designed for Modern Platforms



## 01 Portable

### Standard Across All Implementations

Swap nginx for Cilium or Istio tomorrow. Your HTTPRoute YAML doesn't change — no annotations to rewrite, no controller migration pain across your fleet.

## 02 Multi-Protocol

### HTTP · gRPC · TCP · TLS · UDP

First-class route kinds for every protocol. No custom controller extensions or annotation hacks needed.

## 03 Traffic Splitting

### Canary Deployments Built-In

Native weight-based routing — run 5% canary next to stable with a single `weight` field. No sidecar, no extra controller.

## 04 Role-Oriented

### Ops Owns Gateway · Devs Own Routes

Infra team controls listeners and TLS. App teams manage their own HTTPRoutes. Clean RBAC split by design.

## 05 Cross-Namespace

### Routing via ReferenceGrant

Route to services in other namespaces with explicit trust grants. Secure delegation — no workarounds needed.

## 06 Rich Status

### Accepted · Programmed · ResolvedRefs

Full lifecycle conditions per resource. Know exactly why a route isn't working before you `kubectl` logs.

## 07 Policy Attachment

### TLS, Retries, Timeouts, LB

Extensible policies attach to any resource — Gateway, Route, or Service. Standards-based extensibility without annotation sprawl across your platform.



# Three-Layer Role Model



IAN

Hey, I'm Ian — your Cluster Admin. 🙋

CLUSTER ADMIN | CLUSTER-SCOPED | GatewayClass | istio | nginx | cilium

I provision the GatewayClass — the cluster-wide singleton that picks which controller implementation runs the data plane.

- ▶ Set once for the whole cluster — all namespaces inherit it by name, no per-team config
- ▶ Swap Istio for Cilium? Only I change — zero app YAML touched, full decoupling

Hi! I'm Ana — Platform Engineer. 🛠️

INFRA TEAM | NAMESPACE-SCOPED | Gateway | :443 HTTPS | :80 HTTP | TLS

I manage the Gateway — ports, protocols, TLS certs, and exactly which namespaces can attach routes to my listeners.

- ▶ I control cross-namespace access via allowedRoutes — explicit trust, no implicit wildcards
- ▶ I own TLS termination — app teams reference my Gateway by name, never modify it



ANA



CHIHIRO

Yo! I'm Chihiro — App Developer. 🚀

APP DEV TEAM | NAMESPACE-SCOPED | HTTPRoute | GRPCRoute | TCPRoute

I write Routes that attach to Ana's Gateway — canary splits, header matching, cross-namespace routing — without touching any infra config.

- ▶ Canary deployments with a single `weight:` field — no sidecar, no extra controller
- ▶ Cross-namespace routing via ReferenceGrant — explicit trust grant, Ana keeps data plane control



WHAT'S IN THE BOX

# Route Types & Policy Attachment

## ROUTE TYPES — PICK BY PROTOCOL

**HTTPRoute**

L7 · GA v1

Header match · rewrites · canary splits · redirects

**GRPCRoute**

L7 · GA v1.1

gRPC service + method matching, streaming-aware

**TLSRoute**

L4 · GA v1.5

TLS pass-through — SNI routing, no termination

**TCPRoute**

L4 · Beta

Raw TCP — Postgres, Redis, Kafka, MQTT

**UDPRoute**

Alpha

DNS · DTLS — experimental, not production-ready

**+ Custom**

GEP

Extensible via GEP — bring your own route type

## POLICY ATTACHMENT — NO YAML CHANGES NEEDED

**BackendTLSPolicy**

Gateway

mTLS from gateway to upstream pods

**BackendTrafficPolicy**

HTTPRoute

LB algorithm · session persistence · retries

**ReferenceGrant**

Service

Cross-namespace trust — explicit opt-in only

### WHY POLICY ATTACHMENT?

Attach policies to **existing resources** without modifying their YAML. Infra teams enforce TLS, retries, and load-balancing without touching app team routes.



# Ingress → Gateway API in 4 Steps

Hey **GPEN Bot!** I just heard **Ingress is getting deprecated** in Kubernetes 🐞 I need to move to Gateway API — but I can't afford *any* downtime. How do I migrate safely?



DEV



GPEN BOT

**Great timing!** We've got the perfect tool — meet **ingress2gateway** CNCF — the official CLI that **auto-converts** your Ingress manifests to HTTPRoute YAML. Run *both in parallel* while verifying — **zero downtime, zero risk.**

```
kubectl ingress2gateway print --namespace my-ns
```

That's awesome! 🙌 Can you walk me through the exact steps end-to-end?



DEV



GPEN BOT

## Here's your zero-downtime migration plan! 🚀

### 01 📦 Install Gateway API CRDs

```
kubectl apply -f standard-install.yaml
```

Adds **GatewayClass**, **Gateway**, **HTTPRoute** & **ReferenceGrant** CRDs. Use **experimental-install.yaml** for **TCPRoute** / **GRPCRoute**.

### 02 ⚙️ Deploy GatewayClass + Gateway

```
Istio / Cilium / nginx-gateway
```

Deploy a cluster-scoped **GatewayClass** + namespaced **Gateway** on ports 80/443. Runs alongside existing Ingress — no traffic switch yet.

### 03 🔄 Convert Ingress → HTTPRoute

```
ingress2gateway CLI - run in parallel
```

Auto-generate **HTTPRoute** YAML. Apply it — **Ingress & HTTPRoute both live.** Verify on new Gateway endpoint before cutting over DNS.

### 04 🗑️ Remove Ingress Resources

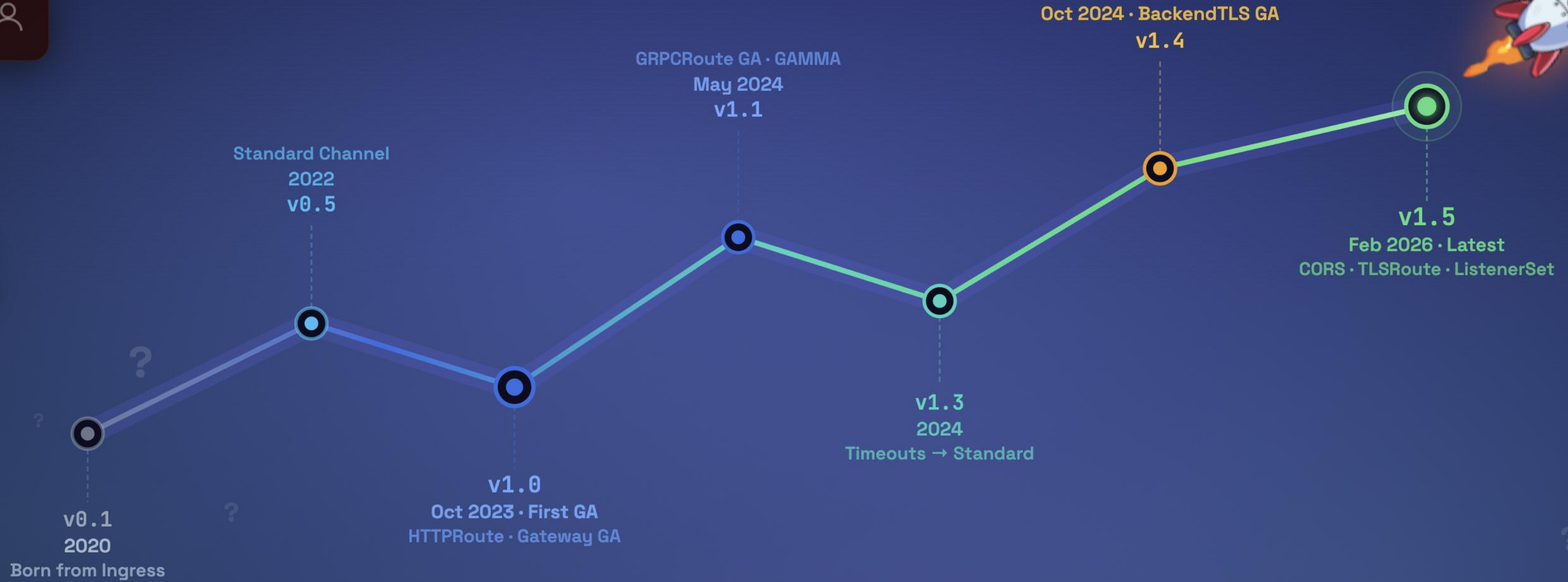
```
zero-downtime cutover ✓
```

`kubectl delete ingress --all -n my-ns`. Update DNS → Gateway LB IP. Decommission old controller. **Done** ✨



# Gateway API Release Timeline

Six years of evolution — from Ingress successor to the **portable, expressive standard** for Kubernetes traffic management.





# Thank You

CONNECT • LEARN • BUILD TOGETHER

